🛠️ PART 0: ENVIRONMENT PREPARATION (Critical)

First (because run all at once is not possible, I've tried)

```
-- 1. Create table structure with explicit Primary Key/Sequence copying

CREATE TABLE rental_large (LIKE rental INCLUDING DEFAULTS INCLUDING
CONSTRAINTS);



-- 2. Seed initial data

INSERT INTO rental_large SELECT * FROM rental;



-- 3. Bulk Insert: Multiply data ~100x (Approx 1.6M rows)

-- We exclude rental_id so the Sequence generates new unique IDs

INSERT INTO rental_large (rental_date, inventory_id, customer_id,
return_date, staff_id)

SELECT rental_date, inventory_id, customer_id, return_date, staff_id

FROM rental

CROSS JOIN generate_series(1, 20);



-- 4. Add UUID and JSONB columns

CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

ALTER TABLE rental_large ADD COLUMN transaction_id UUID DEFAULT
uuid_generate_v4();

 ALTER TABLE rental_large ADD COLUMN meta_data JSONB;
```

Second:

```
-- 5. Populate dummy JSONB data (10% selectivity)

UPDATE rental_large
```

```
SET meta_data = '{"status": "archived", "device": "mobile"}'

 WHERE rental_id % 10 = 0;
```

Third:

```
-- 6. CRITICAL STEP: Update Statistics & Visibility Map

-- Without this, Index Only Scan labs in Part 2 may fail.

 VACUUM (ANALYZE, VERBOSE) rental_large;
```

📝 PART 1: PHYSICAL STORAGE & ADVANCED INDEXING

Task 1: The Ghost in the Machine (TIDs)

Code:

```
SELECT ctid, * FROM rental_large LIMIT 10;
```

Evidence:

Query | Query History | Scratch Pad

```
SELECT ctid, * FROM rental_large LIMIT 10;
```

Data Output | Messages | Notifications

Showing rows: 1 to 10 | Page No: 1 of 1

	ctid tid	rental_id integer	rental_date timestamp without time zone	inventory_id integer	customer_id smallint	return_date timestamp without time zone	staff_id smallint	last_update timestamp without time zone	transaction_id uuid
1	(0,1)	2	2005-05-24 22:54:33	1525	459	2005-05-28 19:40:33	1	2006-02-16 02:30:53	de9a39eb-9fb9-44c0-9859-64c8e87
2	(0,2)	3	2005-05-24 23:03:39	1711	408	2005-06-01 22:12:39	1	2006-02-16 02:30:53	8dc733d5-9445-4493-b341-ef01d3
3	(0,3)	4	2005-05-24 23:04:41	2452	333	2005-06-03 01:43:41	2	2006-02-16 02:30:53	b5c1f9e8-1149-44fe-85b6-fb17c8d
4	(0,4)	5	2005-05-24 23:05:21	2079	222	2005-06-02 04:33:21	1	2006-02-16 02:30:53	819da7c1-a0e5-4021-b620-3ea05b
5	(0,5)	6	2005-05-24 23:08:07	2792	549	2005-05-27 01:32:07	1	2006-02-16 02:30:53	2927f521-6a13-479c-9a11-377efb4
6	(0,6)	7	2005-05-24 23:11:53	3995	269	2005-05-29 20:34:53	2	2006-02-16 02:30:53	41550aef-e680-4441-8af8-396b123
7	(0,7)	8	2005-05-24 23:31:46	2346	239	2005-05-27 23:33:46	2	2006-02-16 02:30:53	d1537bb3-0f4d-40c0-b2d9-3a0e7b
8	(0,8)	9	2005-05-25 00:00:40	2580	126	2005-05-28 00:22:40	1	2006-02-16 02:30:53	955f8944-30dc-4fd8-9b59-8343d75
9	(0,10)	11	2005-05-25 00:09:02	4443	142	2005-06-02 20:56:02	2	2006-02-16 02:30:53	0c65218c-0652-4b27-8bd8-d66469
10	(0,11)	12	2005-05-25 00:19:27	1584	261	2005-05-30 05:44:27	2	2006-02-16 02:30:53	cfea1b43-9972-4aca-a20f-f50a209

Analysis:

The tuple identifier is represented as a pair of numbers in the form (block_number, tuple_index).

The first number indicates the physical page number where the row is stored. PostgreSQL stores table data in fixed-size pages of 8KB, and this number identifies which page on disk contains the row.

The second number represents the position (slot) of the row inside that page. Each page contains multiple row slots, and this value tells PostgreSQL exactly which row to read within the page.

When PostgreSQL needs to access a row, it uses this identifier to directly locate the correct 8KB page using the page number, then accesses the specific row inside the page using the tuple index. This allows PostgreSQL to fetch rows efficiently without scanning the entire table.

This identifier points to a physical location, not a logical one, which is why it can change when rows are updated or moved during maintenance operations.

Task 2: Partial Index Strategy

Code:

```
1.
EXPLAIN ANALYZE SELECT * FROM rental_large WHERE return_date IS NULL;
2.
CREATE INDEX idx_pending_returns ON rental_large(rental_id)
3.
EXPLAIN ANALYZE SELECT * FROM rental_large WHERE return_date IS NULL;
4.
SELECT pg_size_pretty(pg_relation_size('idx_pending_returns'));
```

Evidences:

Before Index:

	QUERY PLAN text
1	Gather (cost=1000.00..7195.55 rows=3717 width=99) (actual time=2.147..84.901 rows=3843 loops=1)
2	Workers Planned: 2
3	Workers Launched: 2
4	-> Parallel Seq Scan on rental_large (cost=0.00..5823.85 rows=1549 width=99) (actual time=0.503..15.624 rows=1281 loo...
5	Filter: (return_date IS NULL)
6	Rows Removed by Filter: 111027
7	Planning Time: 1.024 ms
8	Execution Time: 85.094 ms

After Index:

	QUERY PLAN text
1	Index Scan using idx_pending_returns on rental_large (cost=0.28..3646.46 rows=3717 width=99) (actual time=0.057..2.810 rows=3843 loo...
2	Planning Time: 1.660 ms
3	Execution Time: 2.959 ms

Analysis:

Without the index, PostgreSQL uses a Parallel Sequential Scan. The planning time is shorter because the planner has fewer access paths to evaluate. However, during execution, PostgreSQL must scan a large portion of the table and filter many rows, which results in a high execution time.

After the partial index is created, the planning time becomes slightly longer because the planner must consider and evaluate the index as a possible execution strategy. However, the execution phase is significantly faster because PostgreSQL can directly access only the small subset of rows stored in the index instead of scanning the entire table.

Since execution time dominates the total query cost, the dramatic reduction in execution time outweighs the small increase in planning time. As a result, the overall query runtime is much shorter when the index is used.

The partial index is significantly smaller because it only stores entries for rows that satisfy the condition return_date IS NULL. In this scenario, the index size is only 104KB because it contains only a few hundred active rentals instead of millions of historical records.

A full index would include all rows in the table, including old returned rentals that are never queried again. This increases index size, disk usage, memory consumption, and the number of pages PostgreSQL must scan during query execution.

Because the partial index is smaller and more selective, PostgreSQL can search it faster, perform fewer I/O operations, and produce a more efficient execution plan. This makes the partial index a better choice when queries consistently target a small, well-defined subset of rows, such as active rentals in a table dominated by historical data.

Task 3: JSONB Indexing

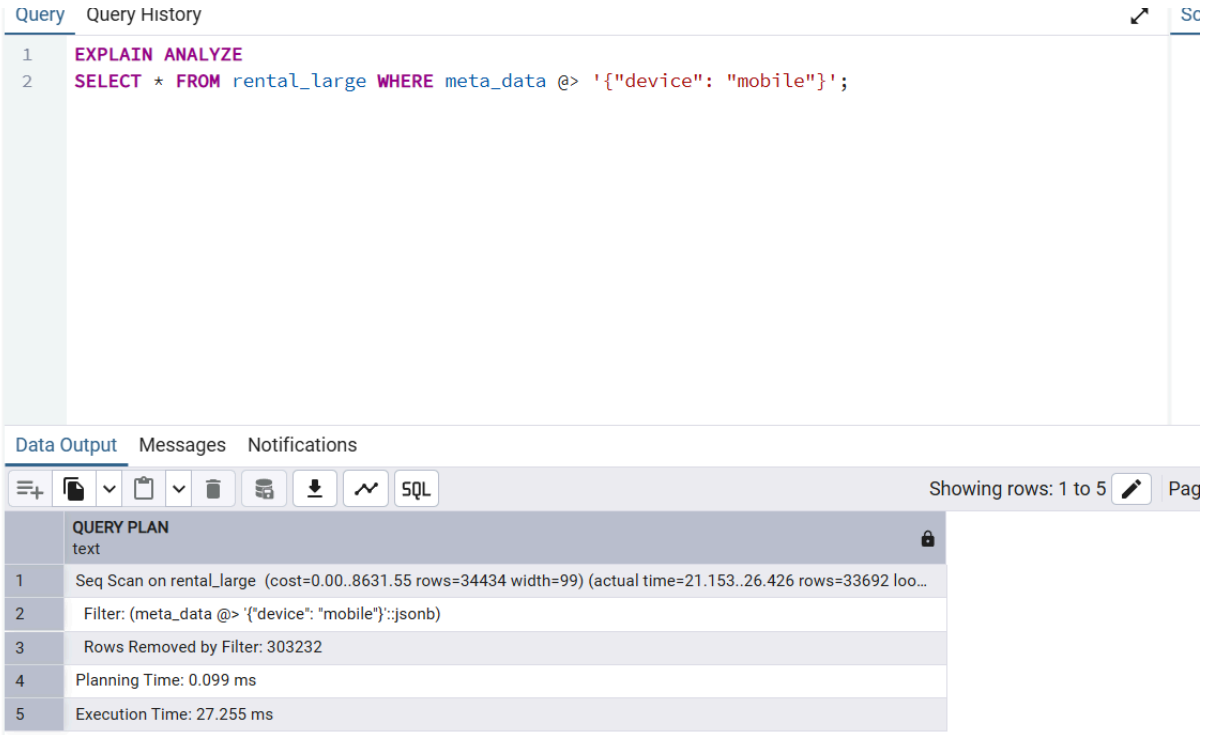
Code:

```
1.
SELECT * FROM rental_large WHERE meta_data @> '{"device": "mobile"}';
```

```
2.
CREATE INDEX idx_meta_data_gin
ON rental_large
USING GIN (meta_data);

3.
SELECT * FROM rental_large WHERE meta_data @> '{"device": "mobile"}';
```

Evidence:



Analysis:

It's Seq Scan

A B-Tree index is designed for ordered and fixed-structure data, where queries compare whole values using operators such as equality or range comparisons. JSON data does not have a fixed schema, and queries may target arbitrary keys and values inside the JSON structure. Because of this, a B-Tree index cannot efficiently index individual JSON elements and is ineffective for containment queries.

A GIN index uses an inverted indexing strategy. Instead of mapping rows to values, it maps individual JSON keys and values to the rows that contain them. This allows PostgreSQL to quickly locate all rows that match a specific JSON condition, such as a particular key-value pair.

When executing the query, PostgreSQL first performs a Bitmap Index Scan using the GIN index to identify which table pages may contain matching rows. It then performs a Bitmap Heap Scan to read only those specific pages from the table. This avoids scanning the entire table and significantly reduces disk I/O, resulting in much better performance for JSON containment queries.

📝 PART 2: THE INDEXING MINDSET

Task 1: The Leading Column Trap

Code:

```
1.
CREATE INDEX idx_wrong_order ON rental_large(rental_date, customer_id);
2.
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM rental_large WHERE customer_id
= 100;
```

```
3.
CREATE INDEX idx_right_order ON rental_large(customer_id, rental_date);
4.
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM rental_large WHERE customer_id
= 100;
```

Evidences

Step 1:

	QUERY PLAN text
1	Bitmap Heap Scan on rental_large (cost=3911.48..5403.58 rows=525 width=99) (actual time=5.506..6.134 rows=504 loops=1)
2	Recheck Cond: (customer_id = 100)
3	Heap Blocks: exact=482
4	Buffers: shared hit=482 read=344
5	-> Bitmap Index Scan on idx_wrong_order (cost=0.00..3911.35 rows=525 width=0) (actual time=5.447..5.447 rows=504 loo…
6	Index Cond: (customer_id = 100)
7	Buffers: shared read=344
8	Planning:
9	Buffers: shared hit=26 read=1 dirtied=2
10	Planning Time: 1.658 ms
11	Execution Time: 6.184 ms

Step 2:

	QUERY PLAN text
1	Bitmap Heap Scan on rental_large (cost=8.49..1500.59 rows=525 width=99) (actual time=0.130..0.800 rows=504 loops…
2	Recheck Cond: (customer_id = 100)
3	Heap Blocks: exact=482
4	Buffers: shared hit=482 read=3
5	-> Bitmap Index Scan on idx_right_order (cost=0.00..8.36 rows=525 width=0) (actual time=0.085..0.085 rows=504 loo…
6	Index Cond: (customer_id = 100)
7	Buffers: shared read=3
8	Planning:
9	Buffers: shared hit=18 read=1
10	Planning Time: 1.577 ms
11	Execution Time: 0.834 ms

Analysis:

In Step 1, the index is defined with rental_date as the leading column and customer_id as the second column. Because the query filters only on customer_id, PostgreSQL cannot directly navigate the index to the matching entries. As a result, it must scan a large portion of the index, build a bitmap of possible matches, and then read many heap pages. This leads to high buffer usage and longer execution time, even though an index is technically used.

In Step 2, customer_id is the leading column of the index. This allows PostgreSQL to quickly locate the exact range of index entries matching the filter condition. The bitmap generated

from the index is much smaller, requiring significantly fewer page reads. Consequently, buffer usage drops dramatically and execution time is greatly reduced.

Although both plans use a Bitmap Scan, the correct index order enables far more selective access. This demonstrates that index column order is critical: an index is only efficient when the query condition matches its leading column, allowing PostgreSQL to minimize I/O and execution cost.

Task 2: I/O Optimization (Covering Index)

Code:

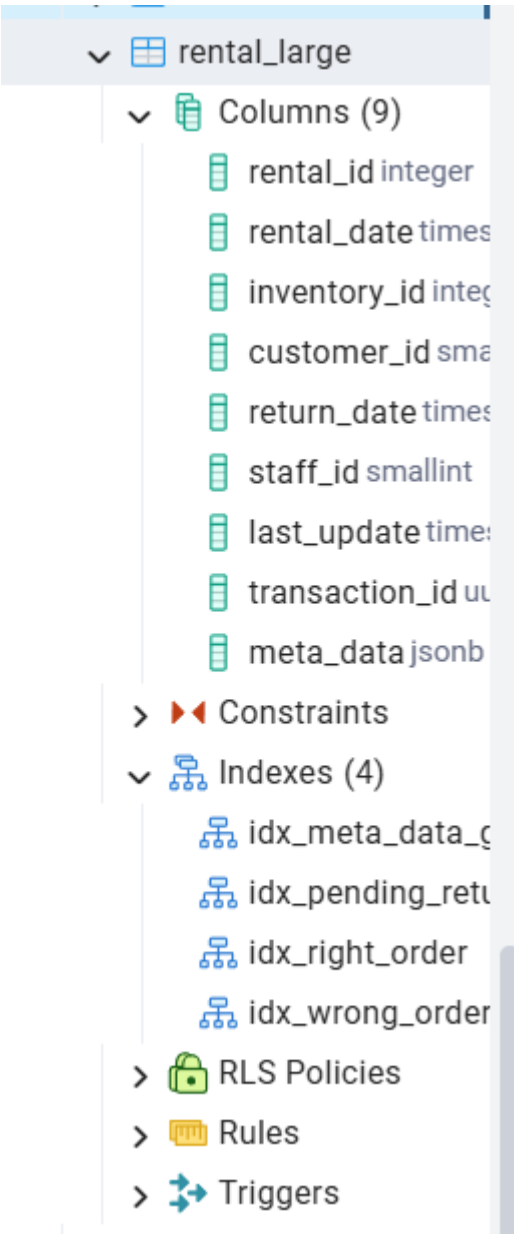
1.

```
EXPLAIN (ANALYZE, BUFFERS) SELECT rental_date FROM rental_large WHERE
customer_id = 100;
```

2.

```
EXPLAIN (ANALYZE, BUFFERS) SELECT rental_date, inventory_id FROM
rental_large WHERE customer_id = 100;
```

Evidences:



It’s still running

Query A:

Query | Query History

```
1  EXPLAIN (ANALYZE, BUFFERS) SELECT rental_date FROM rental_large WHERE customer_id = 100;
```

Data Output | Messages | Notifications

Showing rows: 1 to 8

	QUERY PLAN text
1	Index Only Scan using idx_right_order on rental_large (cost=0.42..25.61 rows=525 width=8) (actual time=0.436..0.486 rows=504 loo…
2	Index Cond: (customer_id = 100)
3	Heap Fetches: 0
4	Buffers: shared hit=7
5	Planning:
6	Buffers: shared hit=90
7	Planning Time: 2.997 ms
8	Execution Time: 0.553 ms

Query B:

Query | Query History

```
1  EXPLAIN (ANALYZE, BUFFERS) SELECT rental_date, inventory_id FROM rental_large WHERE customer_id = 100;
```

Data Output | Messages | Notifications

Showing rows: 1 to 11 | Page No: 1

	QUERY PLAN text
1	Bitmap Heap Scan on rental_large (cost=8.49..1500.59 rows=525 width=12) (actual time=0.229..2.392 rows=504 loops…
2	Recheck Cond: (customer_id = 100)
3	Heap Blocks: exact=482
4	Buffers: shared hit=485
5	-> Bitmap Index Scan on idx_right_order (cost=0.00..8.36 rows=525 width=0) (actual time=0.123..0.123 rows=504 loo…
6	Index Cond: (customer_id = 100)
7	Buffers: shared hit=3
8	Planning:
9	Buffers: shared hit=3
10	Planning Time: 0.198 ms
11	Execution Time: 2.435 ms

Analysis:

Query A uses an Index Only Scan because all columns required by the query are fully contained within the index. PostgreSQL can apply the filter condition and return the selected

column directly from the index without accessing the table. The execution plan confirms this with zero heap fetches, resulting in minimal I/O and faster execution.

Query B cannot use an Index Only Scan because it requests an additional column that is not stored in the index. Although PostgreSQL still uses the index to identify matching rows, it must fetch the missing column from the table. This requires reading heap pages, which increases buffer usage and execution time.

This demonstrates that Index Only Scan is only possible when the index completely covers both the filter and the selected columns, allowing PostgreSQL to avoid reading the table entirely.

📝 PART 3: JOIN ALGORITHMS & PERFORMANCE

Task 1: Algorithm Battle

Code:

1.
```
EXPLAIN (ANALYZE, BUFFERS)
SELECT
   r.rental_id,
   r.rental_date,
   c.customer_id,
   i.inventory_id
FROM rental_large r
JOIN customer c
   ON r.customer_id = c.customer_id
JOIN inventory i
   ON r.inventory_id = i.inventory_id
WHERE r.rental_date >= DATE '2005-06-01'
  AND r.rental_date <  DATE '2005-07-01';
```

2.
```
SET enable_hashjoin = off;
SET enable_mergejoin = off;

EXPLAIN (ANALYZE, BUFFERS)
SELECT
   r.rental_id,
   r.rental_date,
   c.customer_id,
   i.inventory_id
FROM rental_large r
JOIN customer c
   ON r.customer_id = c.customer_id
JOIN inventory i
```

```
    ON r.inventory_id = i.inventory_id
WHERE r.rental_date >= DATE '2005-06-01'
  AND r.rental_date <  DATE '2005-07-01';
```

Evidence:

1.

Query Query History

```sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT
    r.rental_id,
    r.rental_date,
    c.customer_id,
    i.inventory_id
FROM rental_large r
JOIN customer c
    ON r.customer_id = c.customer_id
JOIN inventory i
    ON r.inventory_id = i.inventory_id
WHERE r.rental_date >= DATE '2005-06-01'
  AND r.rental_date <  DATE '2005-07-01';
```

Data Output Messages Notifications

Showing rows: 1 to

	QUERY PLAN text
1	Hash Join (cost=856.24..6267.54 rows=48902 width=20) (actual time=2.113..17.415 rows=48531 loops=1)
2	Hash Cond: (r.inventory_id = i.inventory_id)
3	Buffers: shared hit=766
4	-> Hash Join (cost=728.17..6010.98 rows=48902 width=20) (actual time=1.374..11.046 rows=48531 loops=1)
5	Hash Cond: (r.customer_id = c.customer_id)
6	Buffers: shared hit=741
7	-> Bitmap Heap Scan on rental_large r (cost=705.67..5859.20 rows=48902 width=18) (actual time=1.262..5.041 rows=48531 loop…
8	Recheck Cond: ((rental_date >= '2005-06-01'::date) AND (rental_date < '2005-07-01'::date))
9	Heap Blocks: exact=677
10	Buffers: shared hit=732

2.

Query Query History

```
SET enable_hashjoin = off;
SET enable_mergejoin = off;

EXPLAIN (ANALYZE, BUFFERS)
SELECT
    r.rental_id,
    r.rental_date,
    c.customer_id,
    i.inventory_id
FROM rental_large r
JOIN customer c
    ON r.customer_id = c.customer_id
JOIN inventory i
```

Data Output Messages Notifications

Showing rows: 1 to 31

	QUERY PLAN text
1	Nested Loop (cost=706.25..9853.46 rows=48902 width=20) (actual time=1.844..30.855 rows=48531 loops=1)
2	Buffers: shared hit=7114 read=11
3	-> Nested Loop (cost=705.95..7256.63 rows=48902 width=20) (actual time=0.895..16.289 rows=48531 loops=1)
4	Buffers: shared hit=2502
5	-> Bitmap Heap Scan on rental_large r (cost=705.67..5859.20 rows=48902 width=18) (actual time=0.870..4.175 rows=48531 loops=1)
6	Recheck Cond: ((rental_date >= '2005-06-01'::date) AND (rental_date < '2005-07-01'::date))
7	Heap Blocks: exact=677
8	Buffers: shared hit=729
9	-> Bitmap Index Scan on idx_wrong_order (cost=0.00..693.44 rows=48902 width=0) (actual time=0.821..0.822 rows=48531 loops=...
10	Index Cond: ((rental_date >= '2005-06-01'::date) AND (rental_date < '2005-07-01'::date))

Analysis:

With default settings, PostgreSQL chooses Hash Join because it is more efficient for large result sets. Hash Join works by building in-memory hash tables for the smaller input relations and then scanning the larger relation once to find matching rows. This approach minimizes repeated access and keeps buffer usage low, resulting in shorter execution time.

When Hash Join is disabled, PostgreSQL is forced to use Nested Loop. In this plan, each row from the outer relation triggers repeated lookups on the inner relations. Even with optimizations such as memoization, the number of repeated accesses grows rapidly with the size of the dataset. This leads to significantly higher buffer usage and longer execution time.

Hash Join is generally superior for large datasets because it scales linearly and avoids redundant work. Nested Loop is preferred only for very small result sets or highly selective indexed lookups, where repeated access cost is minimal.

Task 2: Materialized Views

Code:

1.

CREATE VIEW v_stats AS

```
SELECT
   customer_id,
   COUNT(*) AS rental_count
FROM rental_large
GROUP BY customer_id;
EXPLAIN ANALYZE
SELECT *
FROM v_stats
WHERE customer_id = 100;
```

2.

```
CREATE MATERIALIZED VIEW mv_stats AS
SELECT
   customer_id,
   COUNT(*) AS rental_count
FROM rental_large
GROUP BY customer_id;
EXPLAIN ANALYZE
SELECT *
FROM mv_stats
WHERE customer_id = 100;
```

Evidence:

1. Standard View

```
CREATE VIEW v_stats AS
SELECT
    customer_id,
    COUNT(*) AS rental_count
FROM rental_large
GROUP BY customer_id;
EXPLAIN ANALYZE
SELECT *
FROM v_stats
WHERE customer_id = 100;
```

Data Output Messages Notifications

Showing rows: 1

	QUERY PLAN text
1	GroupAggregate (cost=0.42..26.93 rows=1 width=10) (actual time=0.047..0.048 rows=1 loops=1)
2	-> Index Only Scan using idx_right_order on rental_large (cost=0.42..25.61 rows=525 width=2) (actual time=0.013..0.031 rows=504 loo
3	Index Cond: (customer_id = 100)
4	Heap Fetches: 0
5	Planning Time: 0.155 ms
6	Execution Time: 0.078 ms

2. Materialized View

Query Query History

```sql
CREATE MATERIALIZED VIEW mv_stats AS
SELECT
    customer_id,
    COUNT(*) AS rental_count
FROM rental_large
GROUP BY customer_id;
EXPLAIN ANALYZE
SELECT *
FROM mv_stats
WHERE customer_id = 100;
```

Data Output Messages Notifications

	QUERY PLAN text
1	Seq Scan on mv_stats (cost=0.00..36.75 rows=11 width=10) (actual time=0.020..0.062 rows=1 loop...
2	Filter: (customer_id = 100)
3	Rows Removed by Filter: 598
4	Planning Time: 0.295 ms
5	Execution Time: 0.073 ms

Analysis:

A standard view does not store any data; it only stores the query definition. Every time the view is queried, PostgreSQL re-executes the underlying aggregation on the base table. This ensures the result is always up to date but can be slow when operating on large datasets.

A materialized view stores the result of the query physically on disk. Queries against it read precomputed data instead of recomputing the aggregation, which makes selection

significantly faster. However, the stored data does not automatically reflect changes in the base table.

This creates a trade-off between performance and freshness. Materialized views offer superior read performance but are dangerous for real-time data because their contents can become stale unless explicitly refreshed.

📝 PART 4: WINDOW FUNCTIONS (ANALYTICS)

Task 1: Retention Analysis (LAG)

Code:

1.
```
SELECT
    customer_id,
    rental_date,
    rental_date - LAG(rental_date) OVER (
        PARTITION BY customer_id
        ORDER BY rental_date
    ) AS days_between_rentals
FROM rental_large
ORDER BY customer_id, rental_date;
```

Evidence:

public.customer/dvdrental/postgres@PostgreSQL 17

No limit

Query | Query History

```sql
SELECT
    customer_id,
    rental_date,
    rental_date
        - LAG(rental_date) OVER (
            PARTITION BY customer_id
            ORDER BY rental_date
        ) AS days_since_last_rental
FROM rental_large
ORDER BY customer_id, rental_date;
```

Data Output | Messages | Notifications

Showing rows

	customer_id smallint	rental_date timestamp without time zone	days_since_last_rental interval
1	1	2005-05-25 11:30:37	[null]
2	1	2005-05-25 11:30:37	00:00:00
3	1	2005-05-25 11:30:37	00:00:00
4	1	2005-05-25 11:30:37	00:00:00
5	1	2005-05-25 11:30:37	00:00:00
6	1	2005-05-25 11:30:37	00:00:00
7	1	2005-05-25 11:30:37	00:00:00
8	1	2005-05-25 11:30:37	00:00:00

Analysis:

The window function approach processes each row only once within its partition. PostgreSQL scans the data in order, keeps track of the previous rental date for each customer, and computes the difference directly. This single-pass execution grows linearly with the number of rows, resulting in O(N) complexity.

Without window functions, the same problem would require a self-join to match each rental with its previous rental. This approach forces PostgreSQL to compare rows against other rows in the same table, generating large intermediate result sets. As the table grows, the number of comparisons increases rapidly, leading to $O(N^2)$ or higher complexity.

This makes window functions significantly more efficient and scalable for sequential analytical calculations such as retention analysis.

Task 2: The "Running Total" Logic Trap

Code:

1.
```
SELECT
    customer_id,
    payment_date,
    amount,
    SUM(amount) OVER (
        PARTITION BY customer_id
        ORDER BY payment_date
    ) AS running_total
FROM payment;
```

2.
```
SELECT
    customer_id,
    payment_date,
    amount,
    SUM(amount) OVER (
        PARTITION BY customer_id
        ORDER BY payment_date
    ) AS running_total
FROM payment
WHERE running_total > 100;
```

3.
```
WITH payment_with_running_total AS (
    SELECT
        customer_id,
        payment_date,
        amount,
        SUM(amount) OVER (
            PARTITION BY customer_id
            ORDER BY payment_date
        ) AS running_total
    FROM payment
)
SELECT *
FROM payment_with_running_total
WHERE running_total > 100;
```

Evidence:

1.

Query Query History

```sql
SELECT
    customer_id,
    payment_date,
    amount,
    SUM(amount) OVER (
        PARTITION BY customer_id
        ORDER BY payment_date
    ) AS running_total
FROM payment;
```

Data Output Messages Notifications

	customer_id smallint	payment_date timestamp without time zone	amount numeric (5,2)	running_total numeric
1	1	2007-02-14 23:22:38.996577	5.99	5.99
2	1	2007-02-15 16:31:19.996577	0.99	6.98
3	1	2007-02-15 19:37:12.996577	9.99	16.97
4	1	2007-02-16 13:47:23.996577	4.99	21.96
5	1	2007-02-18 07:10:14.996577	4.99	26.95
6	1	2007-02-18 12:02:25.996577	0.99	27.94
7	1	2007-02-21 04:53:11.996577	3.99	31.93
8	1	2007-03-01 07:19:30.996577	4.99	36.92
9	1	2007-03-02 14:05:18.996577	3.99	40.91
10	1	2007-03-02 16:30:04.996577	0.99	41.90
11	1	2007-03-17 11:06:20.996577	4.99	46.89
12	1	2007-03-18 02:25:55.996577	0.99	47.88

2.

Query Query History

```sql
SELECT
    customer_id,
    payment_date,
    amount,
    SUM(amount) OVER (
        PARTITION BY customer_id
        ORDER BY payment_date
    ) AS running_total
FROM payment
WHERE running_total > 100;
```

Data Output Messages Notifications

```
ERROR:  column "running_total" does not exist
LINE 10: WHERE running_total > 100;
               ^


SQL state: 42703
Character: 183
```

3.

Query Query History

```sql
WITH payment_with_running_total AS (
    SELECT
        customer_id,
        payment_date,
        amount,
        SUM(amount) OVER (
            PARTITION BY customer_id
            ORDER BY payment_date
        ) AS running_total
    FROM payment
)
SELECT *
FROM payment_with_running_total
WHERE running_total > 100;
```

Data Output Messages Notifications

	customer_id smallint	payment_date timestamp without time zone	amount numeric (5,2)	running_total numeric
1	1	2007-04-28 07:33:11.996577	4.99	101.75
2	1	2007-04-28 14:46:49.996577	4.99	106.74
3	1	2007-04-28 16:02:05.996577	0.99	107.73
4	1	2007-04-28 17:48:33.996577	0.99	108.72
5	1	2007-04-29 02:27:15.996577	2.99	111.71
6	1	2007-04-30 01:10:44.996577	2.99	114.70
7	2	2007-04-30 12:16:09.996577	10.99	105.78
8	2	2007-04-30 12:42:37.996577	0.99	106.77

Analysis:

The query fails because SQL evaluates the WHERE clause before computing window functions. At the time the WHERE clause is processed, the running total column does not yet exist, since window functions are evaluated after the SELECT phase. As a result, PostgreSQL cannot reference the alias in the WHERE condition.

A Common Table Expression solves this problem by introducing an intermediate execution step. The window function is first evaluated inside the CTE, producing a result set where the running total already exists as a regular column. The outer query can then safely apply filtering conditions to this computed column.

This demonstrates the importance of SQL's logical order of execution and why window function results must be filtered in a separate query layer.